```
void drawHomeScreen() {
  tft.fillScreen(TFT_BLACK);
  tft.setTextColor(TFT_WHITE, TFT_BLACK);

  int buttonWidth = 80, buttonHeight = 60;
  String apps[4] = {"Contacts", "Calculator", "Games", "Timer"};

  // Display buttons for different apps in a grid
  for (int i = 0; i < 4; i++) {
    int x = (i % 2) * (buttonWidth + 10) + 10;
    int y = (i / 2) * (buttonHeight + 10) + 30;

    tft.drawRoundRect(x, y, buttonWidth, buttonHeight, 5, TFT_WHITE);
    tft.setCursor(x + 10, y + 20);
    tft.print(apps[i]);
  }
}
```

Fig. 7: App home screen UI code

```
void loop() {
  if (touch.available()) {
    int x = touch.data.x;
    int y = touch.data.y;

    // Handle touch based on the current screen
    if (currentScreen == HOME) {
      handleHomeScreenTouch(x, y);
    } else if (currentScreen == CALCULATOR) {
      handleCalculatorTouch(x, y);
    } else if (currentScreen == CONTACTS) {
      handleContactTouch(x, y);
    }
  }
```

Fig. 8: Touch input code

```
void drawContacts() {
  tft.fillScreen(TFT_BLACK);

  // Display each contact with a separator line
  for (int i = 0; i < 3; i++) {
    tft.setCursor(20, 40 + i * 40);
    tft.setTextColor(TFT_WHITE, TFT_BLACK);
    tft.print(contacts[i][0]);

    tft.setCursor(20, 60 + i * 40);
    tft.print(contacts[i][1]);

    tft.drawLine(0, 80 + i * 40, 240, 80 + i * 40, TFT_WHITE);  // Draw line separator
  }
}
```

Fig. 9: Contacts code

linked to the GSM module, while voice input is captured from the SMD microphone. Refer to Fig. 6 for details.

Designing app

Next, the app is designed, incorporating basic features, such as contacts, phonebook, timer, games, and calculator. An additional UI screen named 'App' is created, along with sub-UIs for each feature. These invoke the necessary functions and execute the corresponding programs while monitoring incoming calls in the background through threading tasks.

The App home screen displays a list of installed applications, allowing users to select the desired app via touch inputs from the touch panel. For contacts, selecting the relevant app triggers the internal command `AT + CPBR = 1`, which retrieves and displays phone numbers and names on the screen. Similarly, selecting the calculator navigates to the next screen, invoking the calculator app. Fig. 7 shows the App home screen UI code, Fig. 8 presents the touch input code, and Fig. 9 displays the contacts code.

Designing calculator app

For the calculator app design, the same method as the display is employed. However, instead of using the `call.make` function, a custom function performs calculations based on user input and displays the results. The calculator UI features touch buttons for numerical and arithmetic function symbols. At the top of the screen, in green, the input and results of the calculations are presented. The code integrates two functions: one that handles touch input and another that displays